

WEEK : 3 Seller and Inventory Management System

Scenario

The e-commerce system maintains information about sellers and inventory. The system stores seller details and tracks the stock quantity, stock status, and latest update date of products supplied by different sellers.

The system helps manage seller information and monitor product inventory to ensure that product stock and availability can be tracked effectively.

1. Creating Seller and Inventory tables

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7), Seller_Name VARCHAR(25), Email VARCHAR(30), Phone number(10), Address VARCHAR(45));
```

Table created.

```
SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7),  
2 Product_ID VARCHAR(7), Seller_ID VARCHAR(7),  
3 Stock_Quantity number, Stock_status VARCHAR(11),  
4 Last_Updated DATE);
```

Table created.

2. Establish Relationships Using Primary Key

Primary keys are used to uniquely identify records in the Seller and Inventory tables.

- Seller_ID is the primary key of the Seller table.
- Inventory_ID is the primary key of the Inventory table.

The Inventory table is connected to both the Product and Seller tables using foreign keys.

Seller → Inventory

One seller can supply multiple products. Therefore, the relationship between Seller and Inventory is One-to-Many (1:M).

Product → Inventory

A product can have inventory information associated with it. The Inventory table uses Product_ID to identify the corresponding product.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);
```

Table altered.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY(Inventory_ID);
```

Table altered.

3. Foreign key for inventory table:

Foreign Keys

- **Product_ID** in the Inventory table references **Product_ID** in the Product table.
- **Seller_ID** in the Inventory table references **Seller_ID** in the Seller table.

```
SQL> ALTER TABLE Inventory ADD
  2  CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);

Table altered.

SQL> ALTER TABLE Inventory ADD
  2  CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);
```

3. Constraints

Unique Email ID

A UNIQUE constraint is added to the Email column in the Seller table.

This ensures that two sellers cannot have the same email address.

```
SQL> ALTER TABLE Seller ADD
  2  CONSTRAINT UN_mail UNIQUE(Email);

Table altered.
```

Email NOT NULL:

The NOT NULL constraint is applied to the Email column.

This ensures that every seller record contains an email address.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;

Table altered.
```

Add Constraint to Check Stock Quantity is Positive:

The CHECK constraint ensures that the stock quantity in the Inventory table is always greater than zero.

```
SQL> ALTER TABLE Inventory ADD
  2  CONSTRAINT CK_Quan CHECK(Stock_Quantity >=0);

Table altered.
```

5. Inserting Values into Seller Table

```
SQL> INSERT INTO Seller VALUES('&SellerID', '&SellerName', '&email', '&phone', '&Address')
2 ;
Enter value for sellerid: S101
Enter value for sellername: EASYBUY ENTERPRISES
Enter value for email: easybuy111@gmail.com
Enter value for phone: 9012436587
Enter value for address: 1, Anna Nagar, Chennai
old 1: INSERT INTO Seller VALUES('&SellerID', '&SellerName', '&email', '&phone', '&Address')
new 1: INSERT INTO Seller VALUES('S101', 'EASYBUY ENTERPRISES', 'easybuy111@gmail.com', '9012436587', '1, Anna Nagar, Chennai')

1 row created.
```

```
SQL> /
Enter value for sellerid: s104
Enter value for sellername: ateez
Enter value for email: ateezatiny@gmail.com
Enter value for phone: 9087246510
Enter value for address: 34, dubai cross street, madurai
old 1: INSERT INTO Seller VALUES('&SellerID', '&SellerName', '&email', '&phone', '&Address')
new 1: INSERT INTO Seller VALUES('s104', 'ateez', 'ateezatiny@gmail.com', '9087246510', '34, dubai cross street, madurai')

1 row created.
```

```
SQL> /
Enter value for sellerid: s105
Enter value for sellername: kaviya
Enter value for email: kaviyasuresh2@gmail.com
Enter value for phone: 6543209876
Enter value for address: 29, gandhi salai, chennai
old 1: INSERT INTO Seller VALUES('&SellerID', '&SellerName', '&email', '&phone', '&Address')
new 1: INSERT INTO Seller VALUES('s105', 'kaviya', 'kaviyasuresh2@gmail.com', '6543209876', '29, gandhi salai, chennai')

1 row created.
```

```
SQL> INSERT INTO Seller VALUES('&SellerID', '&SellerName', '&email', '&phone', '&Address')
2 ;
Enter value for sellerid: s103
Enter value for sellername: Atoz
Enter value for email: atozmarket143@gmail.com
Enter value for phone: 4343433432
Enter value for address: 23, Thillai nagar, trichy
old 1: INSERT INTO Seller VALUES('&SellerID', '&SellerName', '&email', '&phone', '&Address')
new 1: INSERT INTO Seller VALUES('s103', 'Atoz', 'atozmarket143@gmail.com', '4343433432', '23, Thillai nagar, trichy')

1 row created.
```

```

2
SQL> SELECT * FROM seller;

SELLER_ SELLER_NAME          EMAIL          PHONE
-----
ADDRESS
-----
S101    EASYBUY ENTERPRISES      easybuy111@gmail.com      9012436587
1, Anna Nagar, Chennai

S102    AH & CO              ahseller@gmail.com        6767676767
12,  george twon, chennai

s103    Atoz                   atozmarket143@gmail.com    4343433432
23, Thillai nagar, trichy

SELLER_ SELLER_NAME          EMAIL          PHONE
-----
ADDRESS
-----
s104    ateez                  ateezatiny@gmail.com      9087246510
34, dubai cross street, madurai

s105    kaviya                 kaviyasuresh2@gmail.com    6543209876
29, gandhi salai, chennai

```

6.Inserting Values to Inventory table

```

SQL> INSERT INTO Inventory
  2 VALUES ('Inv0001','P1001','S00001',4,'Available',TO_DATE('2026-09-12','YYYY-MM-DD'));

1 row created.

SQL>
SQL> INSERT INTO Inventory
  2 VALUES ('Inv0002','P1002','S00002',3,'Available',TO_DATE('2026-09-13','YYYY-MM-DD'));

1 row created.

SQL>
SQL> INSERT INTO Inventory
  2 VALUES ('Inv0003','P1003','S00003',4,'Unavailable',TO_DATE('2026-09-15','YYYY-MM-DD'));

1 row created.

SQL>
SQL> INSERT INTO Inventory
  2 VALUES ('Inv0004','P1004','S00004',3,'Available',TO_DATE('2026-09-14','YYYY-MM-DD'));

1 row created.

```

```

SQL> SELECT * FROM Inventory;

INVENTO PRODUCT SELLER_ STOCK_QUANTITY STOCK_STATU LAST_UPDA
-----
Inv001  P100001 S101          4 Available  12-SEP-26
Inv002  P100002 S102          3 Available  13-SEP-26
Inv003  P200001 s103          4 Unavailable 15-SEP-26
Inv004  P200002 s104          3 Available  15-SEP-26

```

7. Report 1: Seller-Wise Product Report

Purpose

This report displays the number of products handled by each seller and the total available stock associated with the seller.

```
SQL> SELECT s.Seller_Name,
2         COUNT(i.Product_ID) AS Number_of_Products,
3         SUM(i.Stock_Quantity) AS Available_Stock
4 FROM Seller s
5 JOIN Inventory i ON s.Seller_ID = i.Seller_ID
6 WHERE i.Stock_Status = 'Available'
7 GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
ateez	1	3
AH & CO	1	3
EASYBUY ENTERPRISES	1	4

8. Report 2: Stock Availability Report

Purpose

This report displays the product name, stock quantity and current stock status.

```
SQL> SELECT p.Product_Name,
2         i.Stock_Quantity,
3         i.Stock_Status
4 FROM Product p
5 JOIN Inventory i ON p.Product_ID = i.Product_ID;
```

PRODUCT_NAME

STOCK_QUANTITY STOCK_STATU

Laptop

4 Available

Head Phones

3 Available

Mobile Phone

4 Unavailable

PRODUCT_NAME

STOCK_QUANTITY STOCK_STATU

Kids Shoes

3 Available

Report 3: Inventory Analysis

The Inventory Analysis report provides information about the overall inventory maintained in the system.

Total Available Products

The following query is used to count inventory records whose stock status is Available.

```
SQL> SELECT COUNT(*) AS Total_Available_Products
  2   FROM Inventory
  3   WHERE Stock_Status = 'Available';
```

```
TOTAL_AVAILABLE_PRODUCTS
-----
                          3
```

Product Out of Stock:

```
SQL> SELECT COUNT(*) AS OutOfStock_Products
  2   FROM Inventory
  3   WHERE Stock_Status = 'Out of Stock';
```

```
OUTOFSTOCK_PRODUCTS
-----
                          0
```

Average Inventory Quantity:

```
SQL> SELECT AVG(Stock_Quantity)AS Average_Inventory FROM Inventory;
```

```
AVERAGE_INVENTORY
-----
                  3.5
```